

Vaporwave Style Graphics Domain Specific Language

Helberth Alejandro, Garcia Melo

Dept. of Computer Engineering

Universidad Distrital Francisco Jose de Caldas

Email: hagarciam@udistrital.edu.co

Abstract—The vaporwave aesthetic, which combines retro visuals with digital art, has gained popularity in modern creative expression, yet generating such graphics usually relies on complex software or AI tools. To address this, we propose a domain-specific language (DSL) that allows users to generate vaporwave-style images using human-readable commands, implemented through a simple compiler built with Python. Our solution successfully generates custom graphics featuring gradient backgrounds, geometric shapes, glitch effects, and typography, demonstrating how DSLs can enable accessible and programmable visual creativity.

Index Terms—Domain-Specific Language (DSL), Compiler, Vaporwave, Image Generation, Python, Syntax Design.

I. INTRODUCTION

The demand for customized graphic content generation has surged in recent years, driven by the proliferation of digital art communities and the growing accessibility of creative tools. Within this creative renaissance, vaporwave has emerged as a prominent aesthetic movement, characterized by nostalgic references to the 1980s and 1990s, pastel and neon color palettes, glitch effects, retro digital typography, and surreal juxtapositions. Despite its popularity and the abundance of tools available for general-purpose graphic design, the creation of vaporwave art still often relies on manual composition and stylistic intuition, lacking automated or domain-specific support that encapsulates its unique visual grammar. To address this gap, this paper proposes the design and implementation of a domain-specific language (DSL) dedicated to the generation of vaporwave-style graphics. DSLs are programming languages tailored to a specific application domain, offering constructs and abstractions that align closely with domain concepts. Prior work on graphic and image generation using DSLs includes Processing [1], a flexible software sketchbook and a language for visual arts based on Java, and Shadertoy, which leverages GLSL for real-time shader development. More recently, domain-specific generative approaches have been explored in systems such as Context Free [2], which uses a grammar-based DSL to generate recursive artistic patterns, and Manim [3], a Python-based library for programmatically generating mathematical animations. These tools, while powerful, are either too general or focused on other visual languages and do not offer abstractions aligned with vaporwave’s visual lexicon. In the field of computational

creativity, techniques such as grammar-based generation, rule-based systems, and procedural modeling have been extensively used to encode stylistic patterns into code [4]. These methods can be harnessed within a DSL to encapsulate vaporwave aesthetics in a structured, reusable, and extensible format. Additionally, recent developments in differentiable graphics and neural rendering [5] suggest future integration opportunities, although this paper focuses on symbolic and procedural approaches to maintain interpretability and user control.

II. METHODS AND MATERIALS

The idea to design this DSL is to offer intuitive, high-level abstractions that reflect the key compositional elements of vaporwave art, including pastel and neon color palettes, retro-futuristic typography, glitch effects, Japanese and Greco-Roman iconography, and grid-based layouts reminiscent of early digital interfaces. To achieve this, the language is implemented as an interpreted, embedded DSL in Python, enabling easy integration with existing graphic generation libraries such as Pillow and Cairo, while maintaining a lightweight syntax focused on artistic expressiveness.

The core objective is to produce a proof-of-concept implementation that enables users to define simple graphic compositions using a small, readable set of custom commands. The DSL will be embedded in Python, allowing us to leverage existing libraries such as Pillow for image manipulation and rendering.

The project will include only the most essential features needed to create typical vaporwave scenes. These features include setting a background color or gradient, placing pre-designed retro-style assets (such as a bust, a sun, or a palm tree), and adding stylized text using nostalgic fonts. The language syntax is intentionally kept simple and declarative. A typical script might include commands like background gradient pink blue, add asset "sun" at center, or text "Aesthetic" at bottom in MS-Gothic. This limited set of commands is sufficient to produce recognizable vaporwave compositions while keeping the language small and the interpreter logic straightforward.

One key design decision is to constrain the visual elements to a fixed canvas size and limit asset placement to a basic grid system. This allows us to avoid dealing with complex coordinate systems while still giving users some creative

control. The MVP will support a few hardcoded templates for layout and colors, which can be extended in future versions.

Although limited in scope, this MVP will demonstrate the feasibility of using a DSL to create stylized art in a specific aesthetic genre. It lays the foundation for future expansion while delivering a fully working and visually coherent prototype. The chosen tools and architecture prioritize simplicity, modularity, and clarity, making the solution suitable for both artistic and educational applications.

For the lexical verification, it was create the following alphabet and tokens:

Language Alphabet

Token	Description	Example
FONDO	Background declaration	fondo
DEGRADADO	Gradient keyword	degradado
COLOR_VALUE	Hexadecimal color code	#FF00FF
COLOR_KEYWORD	Color keyword	color
OPERADOR	Gradient direction operator	->
FIGURA_TYPE	Figure types	sol, piramide
POS	Position of elements	pos(400,300)
TAM	Size declaration	tam 100
TEXTO	Text command	texto
CADENA	String literal	"VAPORWAVE"
EFFECTO_TYPE	Visual effects	scanlines, glitch
GROSOR	Line thickness keyword	grosor 2
ALPHA	Opacity value	alpha 0.5
NUMERO	Number values	100, 2
IGNORAR	Whitespace (ignored)	spaces, tabs

TABLE I

TOKEN ALPHABET OF THE DSL

To provide a robust interpretation of the domain-specific language (DSL) commands, two key components were developed: a lexical analyzer (lexer) and a syntax analyzer (parser). The lexer is responsible for scanning the raw text input and classifying its elements into predefined tokens using regular expressions. These tokens include keywords like `fondo`, `figura`, and `texto`, operators such as `->`, and values like hexadecimal colors, positions, and string literals.

Following lexical analysis, the parser processes the token stream to validate whether the input adheres to the grammatical rules of the language. The parser is implemented using a recursive-descent approach in Python, manually structured to check the correct sequence and nesting of expressions. It ensures, for instance, that a `figura` directive is followed by valid attributes such as `pos`, `tam`, and `color`. If the input is syntactically correct, the parser produces an abstract representation that is used to generate the final image using Pillow.

This two-phase analysis approach improves error detection and separation of concerns: the lexer filters lexical issues, while the parser handles structural correctness. Together, they form the backbone of the DSL's interpretation engine and ensure a clear mapping from human-readable instructions to graphical output.

III. RESULTS

To validate our DSL implementation, we manually executed the core components—lexer, parser, and interpreter—using a

sample `.vapor` script. The code was run locally in a Python environment, and the resulting image was visually inspected to verify correctness. The script produced an output that aligned well with the intended structure of vaporwave art, although some stylistic details such as color palettes and texture variations are yet to be fully refined.

Execution Example

```
fondo degradado #290033 -> #00A2FF
figura sol pos(400,300) tam 100 color #e89d33
texto "VAPORWAVE" pos(300,100) color #FFFFFF
efecto scanlines grosor 2
```

The result is a vaporwave-style image, shown in Figure 1, featuring a vertical gradient background, centered sun figure, retro typography, and scanline effect.

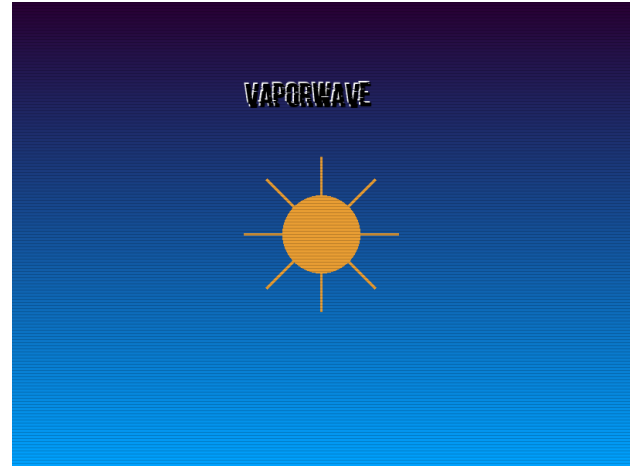


Fig. 1. Image generated using the DSL interpreter

Although the system was not tested using formal unit or integration tests, functional validation was conducted by manually executing code and inspecting visual output. No test framework was used. The DSL design prioritizes readability and declarative control over scene composition. Compared to state-of-the-art AI solutions for image generation, this DSL offers full transparency and deterministic output at the cost of creative flexibility.

IV. CONCLUSIONS

This project aimed to understand how compilers work through the creation of a domain-specific language (DSL) that produces images in the vaporwave aesthetic. The motivation was both technical and artistic: to build a functional compiler pipeline while producing visually appealing outputs that blend retro aesthetics with technological expression. Despite the complexity of implementing lexers, parsers, and interpreters from scratch, the project succeeded in achieving a basic but functional pipeline. The language supports essential features such as background gradients, scanline and glitch effects, and the ability to draw geometric figures like suns and triangles.

It also allows textual elements using a limited set of typographies. Although only one font is currently operational, the output demonstrates the potential of the approach.

The project was partially successful in achieving its artistic goals. While the graphics generated were coherent and followed vaporwave themes, their aesthetic fidelity heavily depended on color choices and was limited by the small set of visual elements supported. The experience also highlighted how much of the heavy lifting in graphics generation can be offloaded to libraries like Pillow, which facilitated rendering. One of the most significant outcomes was recognizing the potential for future improvements: expanding the visual vocabulary, enabling typography variety, integrating image editing capabilities, and potentially making the tool available through a web interface. Although current state-of-the-art vaporwave image generation is dominated by AI models, this open-source DSL offers a transparent and reproducible alternative—possibly even serving as input for future AI pipelines.

REFERENCES

- [1] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys (CSUR)*, 37(4), 316–344. (Base teórica sobre DSLs.)
- [2] Reas, C., & Fry, B. (2007). *Processing: A programming handbook for visual designers and artists*. MIT Press. (DSL para arte visual interactivo.)
- [3] Witkin, A., & Kass, M. (2001). *Context Free Art: A system for rule-based art*. contextfreeart.org (DSL basado en gramáticas para generar imágenes recursivas.)
- [4] Grant Sanderson (3Blue1Brown). *Manim: Mathematical Animation Engine*. <https://github.com/3b1b/manim> (Librería para animación matemática en Python, con DSL interno.)
- [5] Sitzmann, V., Zollhöfer, M., & Wetzstein, G. (2020). *Scene Representation Networks: Continuous 3D-Structure-Aware Neural Scene Representations*. NeurIPS. (Gráficos diferenciables.)
- [6] Y. Espinosa. Desarrollo de un lenguaje de dominio específico (DSL) gráfico para el modelamiento de bases de datos espaciales. [online]. Disponible en: <http://hdl.handle.net/11349/15373>
- [7] Paul Ballam-Cross; Reconstructed Nostalgia: Aesthetic Commonalities and Self-Soothing in Chillwave, Synthwave, and Vaporwave. *Journal of Popular Music Studies* 1 March 2021; 33 (1): 70–93. doi: <https://doi.org/10.1525/jpms.2021.33.1.70>